

Cliplift · Complete Product & Technical Plan

Context

We're building **Cliplift**, a short-form video analytics + publishing tool positioned as the "anti-credit" alternative to Virlo.ai. Virlo's weaknesses: credit-gated pricing, narrow platform coverage, no publishing. Our edge: flat-rate unlimited pricing, insight-to-publish loop, and multi-platform parity.

Data strategy · three-tier model:

The entire competitive analytics market (including Virlo) runs on scraping and third-party data APIs. Virlo literally advertises "Custom Niches scrape TikTok, IG Reels, and YT Shorts 24/7." The API constraint is a shared constraint across the market, not a barrier.

```
TIER 1: OFFICIAL APIs (your own connected accounts · publishing + own analytics)
... YouTube Data API v3           Free, 10K quota/day
... Instagram Graph API          Business accounts only (submit app review in Week 0)
... LinkedIn Marketing API       Publish + own page analytics (apply for partner access)
TIER 2: THIRD-PARTY DATA APIs (competitive intelligence on OTHERS · MVP stage)
... Netrows (LinkedIn + X)        .49/mo, 10K credits, 48+ endpoints
... Data365 (TikTok + IG)        $99/mo, videos/creators/hashtags/trends
... YouTube Data API v3          Free (also covers competitive search)
Total: ~$155/mo for 4-platform competitive intelligence
TIER 3: OWN SCRAPERS (build at >$5K MRR to reduce per-unit cost)
... Playwright + residential proxies
... Build per-platform only when volume justifies it
... Swap in via DataProviderRouter without touching business logic
```

All data sources abstracted behind a **DataProviderRouter** adapter pattern. This is insurance · not over-engineering.

LinkedIn wedge reframed: The moat isn't "we have API access" (anyone can sign up for Netrows). The moat is "we make LinkedIn video data actionable" · benchmarking by industry, cross-platform trend correlation, B2B-specific outlier detection, publishing loop. Data is commodity; the insight layer is the moat.

Infra: FastAPI on Railway (auto-sleep) + QStash for scheduled tasks + Supabase (Postgres) + Upstash Redis + Vercel for frontend. No persistent workers. Timeline: Week 0 setup + 6 build weeks (honest estimate · Week 3 analytics and Week 4 publishing each deserve a full week).

Break-even: ~\$162/mo total costs. Average revenue per user ~\$50/mo. **Break-even at 4 paying users.** Per-user data cost ~\$3/mo · 90% gross margin.

1. Product Vision

One-liner: "Discover what's going viral, publish your response, measure the results · one tool, flat rate, no credits."

Target users (MVP):

- Solo creators (10K-100K followers) posting Shorts + Reels
- Small agencies (5-15 clients) managing short-form across platforms
- B2B growth teams using short-form for brand awareness

Pricing: | Tier | Price | Includes | |-----|-----|-----| | Creator | \$29/mo | Unlimited searches, 1 platform, 1 user, 3 tracked creators | | Team | \$79/mo | Unlimited searches, all platforms, 5 users, 25 tracked creators, scheduling | | Agency | \$149/mo | Everything + API access, 50 tracked creators, white-label reports |

Annual billing: 30% discount. No credits. Ever.

2. MVP Feature Set (Week 0 + 6 build weeks)

In scope (Phase 1 · Launch)

| Feature | Description | |-----|-----| | **Trend Search (PUBLIC)** | Keyword search across YouTube Shorts, TikTok, LinkedIn, Instagram Reels. **No login required** · this is the "time to wow" hook. Paywall starts at tracking/publishing. | | **Outlier Detection** | Algorithm flags videos performing 3x+ above creator's baseline (Z-score). Includes **"Generate idea from this"** button · AI content brief (Claude API, hook analysis + format + CTA pattern) | | **Creator Tracking** | Follow creators, daily metric snapshots (followers, avg views, engagement rate) | | **Video Tracking** | Track specific videos over time (view velocity, engagement curve) | | **Niche Management** | User-defined keyword groups. Auto-discovers trending content within niche | | **Content Calendar** | Visual calendar showing scheduled + published posts | | **Publish to YouTube** | Upload Shorts directly from Cliplift (YouTube Data API) | | **Publish to Instagram** | Publish Reels via Instagram Graph API (business accounts) · contingent on app review approval; read-only analytics until then | | **Analytics Dashboard** | Cross-platform performance overview, creator benchmarks, trend charts | | **LinkedIn Video Analytics** | The differentiator · LinkedIn video performance data nobody else provides. READ-ONLY via Netrows (no publishing). Publishing deferred to Phase 2 (requires Marketing API partnership). | | **TikTok Analytics** | Video search, creator tracking, trend discovery via Data365. READ-ONLY (no publishing). | | **Billing** | Stripe subscriptions, flat-rate plans, self-serve upgrade/downgrade | | **Landing Page** | Conversion-optimized marketing site with pricing + comparison pages |

Out of scope (deferred)

- Full AI content generation (Phase 2) · MVP has "Generate idea from outlier" only
 - Team/collaboration features beyond basic seats (Phase 2)
 - Mobile app
 - White-label reports (Phase 3)
 - In-house scrapers (Phase 3, at \$5K MRR · swap out data providers)
-

3. Tech Architecture

.....			
·	FRONTEND		·
·	Next.js 15 (App Router)		·
·	Tailwind CSS + shadcn/ui + Recharts		·
·	Deployed on Vercel		·
.....			
·	REST API (HTTPS)		·
.....			
·	BACKEND		·
·	FastAPI (Python 3.12) on Railway		·
·			·
·	Supabase Auth	· API v1	· Webhooks
·	(JWT verify)	· Routes	· (Stripe/QSt)
·			·
.....			

.....		
Service Layer		
trend_service, creator_service,		
publishing_service, analytics_service		
.....		
.....		
DataProviderRouter (adapter pattern)		
.....		
YouTube Data365 Netrows		
(official) (TikTok) (Linked)		
.....		
Instagram Swap providers at		
(Graph API) \$5K MRR for scrapers		
.....		
.....		
QStash Worker Endpoints		
/workers/scrape-creators (daily)		
/workers/scrape-videos (6-hourly)		
/workers/discover-trends (hourly)		
/workers/publish-scheduled (5-min)		
/workers/collect-analytics (daily)		
.....		
QStash timeout: 120s for ALL workers		
(publish needs R2 download + YT upload)		
Batch: 1 call per worker, iterates all		
due items (stays within 500 msg/day)		
Publish: 1 post per invocation (large)		
.....		
.....		
Upload Flow (presigned URLs)		
Browser presign req FastAPI		
Browser PUT video Cloudflare R2		
Browser POST /posts {file_key}		
(video never touches our server)		
.....		
.....		
Railway (auto-sleep)		
.....		
.....		
PostgreSQL	Upstash Redis	QStash
(Supabase	(serverless)	(scheduler)
or Neon)		
	Cache	Cron jobs
All data	Rate limits	Retries
	Sessions	Signatures
.....		

Tech Stack Summary

| Layer | Technology | Why | |-----|-----|-----| | Frontend | Next.js 15 (App Router) | SSR for SEO pages, React for dashboard SPA | | UI | Tailwind CSS + shadcn/ui | Fast to build, professional look, zero design cost | | Charts | Recharts | Lightweight, React-native, good for dashboards | | Backend | FastAPI (Python 3.12) | Async, fast, great for data processing | | ORM | SQLAlchemy 2.0 (async) + Alembic | Mature, async-native, reliable migrations | | Task Scheduling | QStash (Upstash) | Serverless cron · HTTP triggers, retries, signatures. No worker processes | Database | PostgreSQL (Supabase/Neon) | Serverless Postgres, generous free tiers | | Cache | Upstash Redis (serverless) | Pay-per-request, no idle cost | | Auth | Supabase Auth (free) | Email/password, Google OAuth, magic links, email verification, password reset · all handled. Backend just validates JWT. Saves 5-7 days vs custom auth. | Email | Resend (free: 3K/mo) | Team invitations, outlier alerts, weekly digests | | Payments | Stripe (subscriptions + webhooks) | Industry standard | | Data: YouTube | YouTube Data API v3 | Free, official, 10K units/day | | Data: TikTok | Data365 API (\$99/mo) | Reliable third-party, no scraper maintenance | | Data: LinkedIn | Netrows API (·49/mo) | Our differentiator · LinkedIn video data nobody else provides | Data: Instagram | Instagram Graph API | Free, official, business accounts | | Frontend hosting | Vercel (free tier) | Perfect for Next.js, global CDN | | Backend hosting | Railway (auto-sleep) | ~\$5/mo, wakes on request, no idle cost |

Data Provider Costs

| Provider | Cost | Data | |-----|-----|-----| | YouTube Data API | Free | 10K units/day (search=100, video=1) | | Data365 (TikTok) | \$99/mo | Videos, creators, hashtags, trends | | Netrows (LinkedIn) | ·49/mo | Video posts, engagement, creator profiles | | Instagram Graph API | Free | Business account analytics + publishing | | **Total data costs** | **~\$155/mo** | 4-platform coverage, zero scraper maintenance |

4. Database Schema

Core Tables

```
-- AUTH & ACCOUNTS
-- users table references Supabase auth.users (no custom auth code)
-- Supabase handles: registration, login, OAuth, email
verification, password reset
profiles (
  id UUID PK REFERENCES auth.users(id),  -- Supabase auth user ID
  email VARCHAR UNIQUE NOT NULL,
  name VARCHAR,
  avatar_url VARCHAR,
  stripe_customer_id VARCHAR,
  created_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ
)
teams (
  id UUID PK,
  name VARCHAR NOT NULL,
  owner_id UUID FK · profiles,
  plan VARCHAR DEFAULT 'creator',      -- creator | team | agency
  stripe_subscription_id VARCHAR,
  max_tracked_creators INT DEFAULT 3,
  max_seats INT DEFAULT 1,
  created_at TIMESTAMPTZ
)
team_members (
  id UUID PK,
```

```

    team_id UUID FK · teams,
    user_id UUID FK · profiles,
    role VARCHAR DEFAULT 'member',          -- owner | admin | member
    invited_at TIMESTAMPTZ,
    joined_at TIMESTAMPTZ
)
-- PLATFORM CONNECTIONS (OAuth tokens for publishing)
platform_connections (
    id UUID PK,
    team_id UUID FK · teams,
    platform VARCHAR NOT NULL,              -- youtube | instagram
    platform_user_id VARCHAR,
    platform_username VARCHAR,
    access_token TEXT,                      -- AES-256 encrypted at app
layer (cryptography.fernet)
    refresh_token TEXT,                    -- AES-256 encrypted at app
layer
    token_expires_at TIMESTAMPTZ,
    scopes VARCHAR[],
    connected_at TIMESTAMPTZ
)
-- CREATOR TRACKING
creators (
    id UUID PK,
    platform VARCHAR NOT NULL,              -- youtube | instagram |
tiktok | linkedin
    platform_id VARCHAR NOT NULL,           -- channel ID / user ID
    username VARCHAR,
    display_name VARCHAR,
    avatar_url VARCHAR,
    bio TEXT,
    is_active BOOLEAN DEFAULT true,
    last_scraped_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ,
    UNIQUE(platform, platform_id)
)
creator_trackings (
    id UUID PK,
    team_id UUID FK · teams,
    creator_id UUID FK · creators,
    tracked_at TIMESTAMPTZ,
    notes TEXT
)
creator_snapshots (
    id UUID PK,
    creator_id UUID FK · creators,
    followers INT,
    total_videos INT,
    avg_views_30d FLOAT,
    avg_engagement_30d FLOAT,              -- (likes+comments) / views
    snapshot_date DATE NOT NULL,
    raw_data JSONB,                        -- full API response
    UNIQUE(creator_id, snapshot_date)
)

```

```

-- VIDEO TRACKING
videos (
  id UUID PK,
  creator_id UUID FK . creators,
  platform VARCHAR NOT NULL,
  platform_video_id VARCHAR NOT NULL,
  title VARCHAR,
  description TEXT,
  thumbnail_url VARCHAR,
  duration_seconds INT,
  published_at TIMESTAMPTZ,
  hashtags VARCHAR[],
  is_short BOOLEAN DEFAULT true,
  -- Creator-relative outlier score (is this above THIS creator's
norm?)
  outlier_score FLOAT,
  is_outlier BOOLEAN DEFAULT FALSE,
  -- Denormalized latest metrics (avoids joins on every dashboard
query)
  latest_views BIGINT DEFAULT 0,
  latest_likes BIGINT DEFAULT 0,
  latest_comments BIGINT DEFAULT 0,
  latest_shares BIGINT DEFAULT 0,
  latest_engagement_rate FLOAT,
  latest_snapshot_at TIMESTAMPTZ,
  last_scraped_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ,
  UNIQUE(platform, platform_video_id)
)
video_snapshots (
  id UUID PK,
  video_id UUID FK . videos,
  views BIGINT,                                -- BIGINT: viral videos
exceed 2.1B (INT max)
  likes BIGINT,
  comments BIGINT,
  shares BIGINT,
  engagement_rate FLOAT,
  view_velocity FLOAT,                          -- views per hour since last
snapshot
  snapshot_at TIMESTAMPTZ,
  raw_data JSONB
)
video_trackings (
  id UUID PK,
  team_id UUID FK . teams,
  video_id UUID FK . videos,
  tracked_at TIMESTAMPTZ
)
-- NICHE / TREND DISCOVERY
niches (
  id UUID PK,
  team_id UUID FK . teams,
  name VARCHAR NOT NULL,

```

```

    keywords VARCHAR[] NOT NULL,          -- search terms
    platforms VARCHAR[] DEFAULT
' {youtube,instagram,linkedin,tiktok}',
    is_active BOOLEAN DEFAULT true,
    last_analyzed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ
)
niche_videos (
    id UUID PK,
    niche_id UUID FK · niches,
    video_id UUID FK · videos,
    outlier_score FLOAT,                  -- Z-score vs niche baseline
    discovered_at TIMESTAMPTZ
)
-- PUBLISHING & SCHEDULING
scheduled_posts (
    id UUID PK,
    team_id UUID FK · teams,
    connection_id UUID FK · platform_connections,
    created_by UUID FK · profiles,        -- who created (team audit
trail)
    inspired_by_video_id UUID FK · videos, -- the outlier that
inspired this post (insight·action link)
    platform VARCHAR NOT NULL,
    title VARCHAR,
    description TEXT,
    hashtags VARCHAR[],
    file_key VARCHAR,                    -- R2 object key from
presigned upload
    media_url VARCHAR,                  -- full URL (generated from
file_key at publish time)
    scheduled_for TIMESTAMPTZ NOT NULL,
    status VARCHAR DEFAULT 'draft',      -- draft | scheduled |
publishing | published | failed
    platform_post_id VARCHAR,           -- ID after publishing
    error_message TEXT,
    published_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ
)
-- ANALYTICS (for user's own published content)
post_analytics (
    id UUID PK,
    post_id UUID FK · scheduled_posts,
    views INT,
    likes INT,
    comments INT,
    shares INT,
    watch_time_seconds INT,
    avg_view_duration FLOAT,
    snapshot_at TIMESTAMPTZ,
    raw_data JSONB
)

```

Indexes (critical for performance)

```
CREATE INDEX idx_videos_platform_published ON videos(platform,
published_at DESC);
CREATE INDEX idx_video_snapshots_video_time ON
video_snapshots(video_id, snapshot_at DESC);
CREATE INDEX idx_creator_snapshots_creator_date ON
creator_snapshots(creator_id, snapshot_date DESC);
CREATE INDEX idx_niche_videos_niche_score ON niche_videos(niche_id,
outlier_score DESC);
CREATE INDEX idx_scheduled_posts_team_status ON
scheduled_posts(team_id, status, scheduled_for);
CREATE INDEX idx_creators_platform ON creators(platform,
platform_id);
CREATE INDEX idx_videos_outlier ON videos(platform, is_outlier,
latest_engagement_rate DESC);
CREATE INDEX idx_videos_creator_views ON videos(creator_id,
latest_views DESC);
```

5. Project Structure (Monorepo)

```
cliplift/
... backend/
.   ... app/
.   .   ... main.py           # FastAPI app factory
.   .   ... config.py        # Pydantic BaseSettings
.   .   ... database.py      # Async engine + session
.   .   ... dependencies.py  # get_db, get_current_user
.   .   .
.   .   ... auth/            # Auth (thin layer · Supabase
handles heavy lifting)
.   .   .   ... middleware.py # Validate Supabase JWT, extract
user
.   .   .   ... models.py     # Profile model (extends auth.users)
.   .   .   ... schemas.py    # Profile response schemas
.   .   .
.   .   ... creators/        # Creator tracking domain
.   .   .   ... routes.py
.   .   .   ... schemas.py
.   .   .   ... models.py
.   .   .   ... service.py
.   .   .
.   .   ... videos/         # Video tracking domain
.   .   .   ... routes.py
.   .   .   ... schemas.py
.   .   .   ... models.py
.   .   .   ... service.py
.   .   .
.   .   ... discovery/      # Trend search & niche analysis
.   .   .   ... routes.py
.   .   .   ... schemas.py
.   .   .   ... models.py
```



```

. . . ... service.py
. . . ... outlier.py # Z-score outlier detection
. . .
. . . ... publishing/ # Scheduling & publishing
. . . ... routes.py
. . . ... schemas.py
. . . ... models.py
. . . ... service.py
. . .
. . . ... analytics/ # Dashboard analytics
. . . ... routes.py
. . . ... schemas.py
. . . ... service.py
. . .
. . . ... billing/ # Stripe integration
. . . ... routes.py
. . . ... schemas.py
. . . ... service.py
. . .
. . . ... platforms/ # DataProviderRouter + adapters
. . . ... base.py # Abstract DataProvider interface
. . . ... router.py # DataProviderRouter (adapter
registry)
. . . ... youtube.py # YouTube Data API v3 (free)
. . . ... tiktok.py # Data365 API ($99/mo)
. . . ... linkedin.py # Netrows API (.49/mo)
. . . ... instagram.py # Instagram Graph API (free)
. . .
. . . ... workers/ # QStash-triggered HTTP endpoints
. . . ... middleware.py # QStash signature verification
. . . ... scrape_creators.py # POST /workers/scrape-creators
. . . ... scrape_videos.py # POST /workers/scrape-videos
. . . ... discover_trends.py # POST /workers/discover-trends
. . . ... publish_post.py # POST /workers/publish-scheduled
. . . ... collect_analytics.py # POST /workers/collect-analytics
. . .
. . . ... alembic/ # Database migrations
. . . ... env.py
. . . ... versions/
. . . ... alembic.ini
. . . ... requirements.txt
. . . ... Dockerfile
. . . ... pytest.ini
.
... frontend/
. . . ... app/ # Next.js App Router
. . . ... (marketing)/ # Public pages (route group)
. . . ... page.tsx # Landing page
. . . ... pricing/page.tsx
. . . ... compare/virlo/page.tsx # SEO comparison page
. . . ... (auth)/ # Auth pages
. . . ... login/page.tsx
. . . ... register/page.tsx
. . . ... (dashboard)/ # Authenticated app

```

```

. . . ... layout.tsx           # Sidebar + nav
. . . ... dashboard/page.tsx   # Overview
. . . ... discover/page.tsx    # Trend search
. . . ... creators/
. . . .   ... page.tsx         # Creator list
. . . .   ... [id]/page.tsx    # Creator detail
. . . ... niches/
. . . .   ... page.tsx
. . . .   ... [id]/page.tsx
. . . ... schedule/page.tsx    # Content calendar
. . . ... analytics/page.tsx   # Performance
. . . ... settings/
. . . .   ... page.tsx        # Account
. . . .   ... team/page.tsx
. . . .   ... billing/page.tsx
. . . .   ... connections/page.tsx
. . . ... layout.tsx          # Root layout
. . . ... globals.css
. . . ... components/
. . . .   ... ui/              # shadcn/ui components
. . . .   ... charts/          # Recharts wrappers
. . . .   ... forms/           # Form components
. . . .   ... layout/          # Navigation, sidebar, etc.
. . . ... lib/
. . . ... api.ts              # Backend API client (fetch
wrapper)
. . . ... auth.ts             # Token management
. . . ... utils.ts
. . . ... hooks/              # Custom React hooks
. . . ... types/              # TypeScript interfaces
. . . ... tailwind.config.ts
. . . ... next.config.ts
. . . ... package.json
. . . ... Dockerfile
.
... docker-compose.yml        # Local dev: postgres + redis
... .env.example
... Makefile                  # dev, migrate, worker, test
shortcuts
... STRATEGY.md               # Competitive strategy
(existing)
... README.md

```

6. API Design

Auth (handled by Supabase · no custom endpoints)

```

# Frontend calls Supabase Auth SDK directly:
# supabase.auth.signUp(), signInWithPassword(),
signInWithOAuth({provider: 'google'})
# supabase.auth.signOut(), resetPasswordForEmail(), etc.

```

```
#
# Backend validates Supabase JWT on every request:
#   Authorization: Bearer <supabase_jwt>
#   · dependencies.py: get_current_user() validates token, returns
  user
GET    /api/v1/profile                · current user profile
PUT    /api/v1/profile                { name, avatar_url } · update
profile
```

Pagination (all list endpoints use cursor-based pagination)

```
# Standard query params on all list endpoints:
?limit=20&cursor=<base64_encoded_cursor>
# Response shape:
{
  "items": [...],
  "next_cursor":
  "eyJjcmVhdGVkX2F0IjoiMjAyNi0wNC0wMyIsImVklkIjoiYWJjIn0=",
  "has_more": true
}
```

Platform Connections

```
GET    /api/v1/connections            · list connected accounts
POST   /api/v1/connections/{platform}/authorize · OAuth redirect URL
GET    /api/v1/connections/{platform}/callback  · OAuth callback
DELETE /api/v1/connections/{id}
```

Trend Discovery (PUBLIC · no auth required for search, paywall on tracking/publishing)

```
POST   /api/v1/discover/search        { query, platforms[],
sort_by, date_range } · PUBLIC, rate-limited
GET    /api/v1/discover/trending      ?platform=&niche_id=&limit=
· PUBLIC
GET    /api/v1/discover/outliers      ?platform=&min_score=3.0
· PUBLIC
POST   /api/v1/discover/generate-idea { video_id } · AI content
brief · AUTH REQUIRED
```

Creators

```
GET    /api/v1/creators              · tracked creators list
POST   /api/v1/creators/track        { platform, platform_id OR
url }
DELETE /api/v1/creators/{id}/untrack
GET    /api/v1/creators/{id}         · creator detail + recent
snapshots
GET    /api/v1/creators/{id}/videos  · creator's videos sorted by
performance
```

Videos

GET	/api/v1/videos	· tracked videos list
POST	/api/v1/videos/track	{ platform, url }
GET	/api/v1/videos/{id}	· video detail + snapshots
over time		
GET	/api/v1/videos/{id}/snapshots	· time-series performance data

Niches

GET	/api/v1/niches	· user's niches
POST	/api/v1/niches	{ name, keywords[], platforms[] }
PUT	/api/v1/niches/{id}	
DELETE	/api/v1/niches/{id}	
GET	/api/v1/niches/{id}/feed	· latest videos in niche, sorted by outlier score

Uploads (presigned URLs · video never touches our server)

POST	/api/v1/uploads/presign	{ filename, content_type } · { upload_url, file_key }
# Browser PUTs video directly to Cloudflare R2 via upload_url		
# Then references file_key when creating a post		

Publishing

GET	/api/v1/posts	· scheduled + published posts
POST	/api/v1/posts	{ platform, connection_id, title, description, file_key, scheduled_for, inspired_by_video_id? }
PUT	/api/v1/posts/{id}	
DELETE	/api/v1/posts/{id}	
POST	/api/v1/posts/{id}/publish-now	
GET	/api/v1/posts/calendar	?month=2026-04 · calendar view data

Analytics

GET	/api/v1/analytics/overview	· dashboard summary (total views, engagement, top posts)
GET	/api/v1/analytics/posts	· performance of published content
GET	/api/v1/analytics/trends	· trend performance over time

Billing

POST	/api/v1/billing/checkout	{ plan } · Stripe Checkout Session URL
GET	/api/v1/billing/subscription	· current plan + usage
POST	/api/v1/billing/portal	· Stripe Customer Portal URL
POST	/api/v1/billing/webhooks	(Stripe webhook endpoint)

7. Core Algorithms

Outlier Detection (the "magic")

```
# For each video in a niche search result:
# 1. Calculate baseline = median views for videos in that niche
#    (last 30 days)
# 2. Calculate std_dev of views in niche
# 3. outlier_score = (video_views - baseline) / std_dev (Z-score)
# 4. Flag videos with outlier_score >= 3.0 (3 standard deviations
#    above mean)
#
# This is Virlo's "Outlier Analysis" · but ours runs on every search,
# unlimited.
```

View Velocity Tracking

```
# For tracked videos, snapshot every 6 hours:
# velocity = (current_views - previous_views) / hours_elapsed
# Acceleration = current_velocity - previous_velocity
# Flag "accelerating" videos (velocity increasing = going viral
# NOW)
```

8. Week-by-Week Build Plan

Week 0: Pre-Sprint Setup (DO BEFORE WRITING CODE)

- [] **Submit Meta App for Instagram review** · this takes 4-8 weeks, start NOW
 - Register Meta Developer account
 - Create Meta App
 - Submit Business Verification documents
 - Request `instagram_content_publish` + `instagram_basic` permissions
 - Write privacy policy and data deletion callback URL
- [] Register YouTube Data API key (Google Cloud Console)
- [] Sign up for Netrows API (LinkedIn data, ~49/mo)
- [] Sign up for Data365 API (TikTok data, \$99/mo)
- [] Create Supabase project (PostgreSQL + Auth)
- [] Install Supabase CLI (`npmx supabase init` + `npmx supabase start` for local dev · replaces raw docker postgres)
- [] Create Upstash account (Redis + QStash)
- [] Purchase domain
- [] Set up Stripe account (test mode)
- [] Set up Resend account (free tier, configure domain DNS)
- [] **Deliverable:** All external accounts ready, Instagram review submitted, `supabase start` running locally

Week 1: Foundation + DataProviderRouter

- [] Project scaffolding (monorepo, local dev via `supabase start` + docker redis)

- [] FastAPI app setup (config, database, async engine, /health warm-up endpoint, **CORS middleware from day 1**)
- [] SQLAlchemy models + Alembic initial migration (all tables, BIGINT for metrics, **profile creation trigger on auth.users INSERT**)
- [] Supabase Auth integration: JWT validation middleware, profile model (no custom auth code)
- [] Cursor-based pagination utility (reusable across all list endpoints)
- [] API rate limiting via Upstash Redis (**upstash-ratelimit**, 30 req/min/user on search endpoints)
- [] **DataProviderRouter** + abstract DataProvider interface
- [] YouTube adapter (YouTube Data API v3 · search, video details, channel details)
- [] LinkedIn adapter (Netrows API · video search, creator profiles, engagement)**DAY 1**
- [] **DIFFERENTIATOR**
- [] TikTok adapter (Data365 API · video search, creator profiles, trends)
- [] Instagram adapter · READ-ONLY (Data365, no Graph API needed for analytics)
- [] Next.js setup (App Router, Tailwind, shadcn/ui)
- [] Supabase Auth UI: login, register, Google OAuth (using @supabase/auth-ui-react or custom)
- [] App shell: sidebar navigation, layout, placeholder pages
- [] **Deliverable:** User can register, log in, see empty dashboard. All 4 platform adapters return data.

Week 2: Trend Discovery Engine + QStash Workers

- [] QStash setup (middleware for signature verification, cron schedules, **120s timeout for all workers**)
- [] Search endpoint (query · DataProviderRouter · all platforms · scored **paginated**)
- [] Outlier detection algorithm (Z-score: creator-relative on **videos**, niche-relative on **niche_videos**)
- [] Creator tracking: track/untrack + QStash daily snapshot worker
- [] Video tracking: track/untrack + QStash 6-hour snapshot worker
- [] Niche management: CRUD + QStash auto-discovery worker
- [] OAuth flow for YouTube + Instagram connections (for publishing later)
- [] Discover page UI (search bar, results grid, outlier badges, platform filters) · **PUBLIC, no auth required** (paywall on track/publish)
- [] Creator list + detail page UI
- [] **Deliverable:** User can search trends across 4 platforms, see outliers, track creators

Week 3: Analytics Dashboard

- [] Dashboard overview endpoint (aggregate metrics across platforms)
- [] Creator snapshots · time-series chart data
- [] Video snapshots · performance curves + view velocity
- [] Niche feed endpoint (videos sorted by outlier score)
- [] Dashboard UI (metric cards, charts, trend feed)
- [] Creator detail UI (follower chart, video grid, engagement trends)
- [] Niche detail UI (keyword performance, top videos)
- [] "Generate idea from this outlier" button · Claude API content brief (hook, format, CTA analysis)
- [] **Deliverable:** Full analytics dashboard with real data + AI content briefs on outliers

Week 4: Publishing & Scheduling

- [] **Presigned upload flow:** POST /uploads/presign · browser uploads to R2 directly · **file** stored
- [] YouTube Shorts upload via API (videos.insert, from R2 **file_key**)
- [] Instagram Reels publish via Graph API (IF approved; YouTube-only fallback)

- [] Scheduled post CRUD endpoints + `inspired_by_video_id` + `created_by` audit
- [] QStash publish worker (**120s timeout** · downloads from R2 then uploads to YouTube/Instagram, one post per invocation for large files)
- [] Content calendar UI (month/week view)
- [] Post composer UI (upload video via presigned URL, write caption, pick time, select platform, link "inspired by" outlier)
- [] Post status tracking (draft · scheduled · publishing · published/failed)
- [] **Deliverable:** User can schedule and publish Shorts (+ Reels if approved). Full insight-to-action loop complete.

Week 5: Billing, Polish & Landing

- [] Stripe integration (checkout, subscription webhooks, portal)
- [] Plan enforcement middleware (check tracked creator limits, seat limits per tier)
- [] Billing settings UI (current plan, upgrade, invoices)
- [] Team settings UI (invite members, manage roles)
- [] Platform connections settings page
- [] Landing page (hero, features, social proof, pricing table)
- [] `/compare/virlo` SEO comparison page
- [] Error handling, loading states, empty states, toast notifications
- [] **Deliverable:** Production-ready app with billing

Week 6: Deploy & Launch

- [] Railway deployment (FastAPI only · auto-sleep)
- [] Vercel deployment (Next.js frontend)
- [] QStash cron schedules configured (all worker endpoints)
- [] Domain setup + SSL (CORS already configured in Week 1)
- [] Monitoring: Sentry (errors), basic logging
- [] Seed 10 niches with real data for demo experience
- [] Invite 10·20 beta creators (free access)
- [] Product Hunt launch prep
- [] "Virlo Alternative" blog posts (SEO)
- [] Creator partnership outreach
- [] Reddit/Discord community posts
- [] **Deliverable:** Live product, first users onboarded

Instagram Contingency Plan

- IF Instagram app review approved by Week 3:
 - Launch with YouTube + Instagram publishing
 - Landing page: "Publish to YouTube & Instagram"
- IF NOT approved by Week 3:
 - Launch with YouTube publishing only
 - Instagram remains READ-ONLY (analytics via data providers)
 - Landing page: "Track across YouTube, Instagram, LinkedIn, TikTok. Publish to YouTube. Instagram publishing coming soon."
 - This is STILL stronger than Virlo (they don't publish at all)

9. Infrastructure & Costs

Local Development

```
supabase start          · Postgres + Auth + Realtime on localhost
(replaces raw docker postgres)
docker-compose.yml:
  - redis:7 (port 6379)    · for caching/rate-limiting (or use
Upstash free tier directly)
Backend: uvicorn (port 8000)
Frontend: next dev (port 3000)
QStash: Use Upstash directly (free tier for dev)
```

Production (Managed Services)

Service	Provider	Cost/mo	Notes
FastAPI (web)	Railway (auto-sleep)	~\$5	Wakes on request, no idle cost
PostgreSQL	Supabase (free tier)	\$0	500MB, auto-backups, connection pooling
Redis	Upstash (free tier)	\$0	10K commands/day free
QStash	Upstash (free tier)	\$0	500 messages/day free
Next.js frontend	Vercel (free tier)	\$0	
Domain	Cloudflare	~\$1	
File storage	Cloudflare R2	~\$0.5	
Sentry	Free tier	\$0	
Resend (email)	Free tier	\$0	3K emails/mo · invites, alerts, digests
Data365 (TikTok)	\$99		Third-party data
Netrows (LinkedIn)	Netrows	~\$53	Third-party data
Total		~\$160/mo	Infra: ~\$6. Data providers: ~\$155.

API Quotas (free tiers)

API	Free Limit	Enough for MVP?
YouTube Data API v3	10,000 units/day	Yes (search=100 units, video detail=1 unit)
Instagram Graph API	200 calls/hr per user	Yes (if approved; read-only until then)
Stripe	No platform fee until revenue	Yes
Cloudflare R2	10GB storage, 10M reads	Yes
Upstash Redis	10K commands/day	Yes for MVP
Upstash QStash	500 messages/day	Yes for MVP
Supabase Postgres	500MB, unlimited API	Yes for first ~1000 users

10. Risks & Mitigations

Risk	Impact	Mitigation
Instagram app review not approved by Week 3	No Instagram publishing at launch	Already mitigated: Submit in Week 0. Instagram analytics still works via data providers (read-only). YouTube publishing is primary. Virlo doesn't publish at all · we still ahead.
YouTube API quota hit at scale	Search stops working	Aggressive caching (Upstash Redis, 1hr TTL). Request quota increase from Google.
Data365 or Netrows goes down	TikTok or LinkedIn data unavailable	DataProviderRouter adapter pattern means we swap providers without touching business logic. Multiple providers exist for each platform.
Data provider price increase	Margin squeeze	At \$5K MRR, build in-house scrapers and swap out providers. Adapter pattern makes this a drop-in replacement.
Low initial traction	No revenue	Free tier generates SEO content (comparison pages). Creator partnerships for social proof. LinkedIn angle has zero competition · organic discovery.
Virlo adds publishing	Differentiation shrinks	Our flat-rate pricing moat is structural · they can't rebuild their credit system.
LinkedIn coverage is our wedge	they can't copy (they're TikTok-centric).	

11. Verification & Testing Plan

Testing during development

- Backend: pytest + httpx for API tests, factory_boy for fixtures
- Frontend: Playwright for critical flows (register · search · schedule · publish)
- Run `supabase start && docker compose up -d redis && pytest` · all green before each week's milestone

End-to-end verification at launch

- 1 Register a new account
 - 2 Connect a YouTube channel
 - 3 Search "cooking shorts" · see results with outlier scores
 - 4 Track a creator · verify daily snapshot appears next day
 - 5 Create a niche "fitness" · see auto-discovered videos
 - 6 Schedule a Short · verify it publishes at the right time
 - 7 Check analytics dashboard · confirm metrics update
 - 8 Subscribe to Team plan via Stripe · verify access to all features
 - 9 Invite a team member · verify they can access the dashboard
-

12. Honest Moat Assessment

With third-party APIs, data access itself is NOT a moat. Anyone can sign up for Netrows. So what IS our moat?

Real moats (defensible)

- 1 **Accumulated snapshot data.** Every day we run, we accumulate historical trend data. A competitor starting 6 months later has 6 months less history. This compounds.
- 2 **AI enrichment quality.** Raw metrics are commodity. Our outlier scoring, hook analysis, topic clustering, and cross-platform trend correlation get better with more data. The algorithm IS the product.
- 3 **Publishing + measurement loop.** Requires OAuth integrations per platform. 2-4 weeks engineering per platform. Most analytics-only tools won't build this because it changes their product category.
- 4 **Switching cost via workflow integration.** Once a team's content calendar, alert rules, and tracked creators live in Cliplift, switching means rebuilding manually.

Not moats (but still advantages)

- Flat-rate pricing (easily copied, but Virlo hasn't changed in 18+ months)
 - LinkedIn coverage (gettable by competitors, but nobody has bothered)
 - Free tools for SEO (commodity tactic, but effective for acquisition)
-

13. Provider Dependency Playbook

Scenario: Netrows raises prices from ·49 · ·249/mo, breaking unit economics.

Playbook:

- 1 **Immediate:** Switch to alternative provider (Phyllo, ProxyCurl for LinkedIn, SociaVault for multi-platform)
- 2 **Short-term (2 weeks):** Build Playwright scraper for LinkedIn public profiles
- 3 **Long-term:** Own scraper + proxy infrastructure for highest-volume platforms

Trigger to build own scrapers:

```
IF monthly_data_api_spend > $500/mo
  AND MRR > $5,000
  AND specific_platform_volume > 10,000 requests/mo
```

```
THEN build own scraper for THAT platform only, keep third-party as
fallback
```

Third-Party Data API Comparison (backup providers)

Provider	Platforms	Pricing	LinkedIn?	TikTok?	Instagram?	YouTube?	Twitter?	Facebook?	Other
Netrows (primary)	LinkedIn, X, Crunchbase	\$49/mo (10K credits)	Yes (48+ endpoints)	No	No	No	No	No	Data365 (primary) TikTok, Instagram, Facebook \$49-99/mo No Yes Phyllo (backup) LinkedIn, Instagram, YouTube, TikTok Custom Yes Yes SociaVault (backup) TikTok, Instagram, LinkedIn, YouTube Pay-per-use Yes Yes EnsembleData (backup) TikTok, Instagram, YouTube Custom No Yes

Files to Create First

- ```

1 docker-compose.yml · local dev services (postgres + redis)
2 backend/app/main.py · FastAPI app factory + CORS + health endpoint
3 backend/app/config.py · Pydantic BaseSettings (all API keys, DB URLs)
4 backend/app/database.py · async SQLAlchemy engine + session
5 backend/app/auth/middleware.py · Supabase JWT validation (thin layer)
6 backend/app/dependencies.py · get_db, get_current_user, pagination params
7 backend/app/platforms/base.py · DataProvider abstract interface
8 backend/app/platforms/router.py · DataProviderRouter
9 backend/app/platforms/youtube.py · first adapter (free, instant)
10 backend/app/auth/models.py · Profile + Team models
11 backend/alembic/ · migration setup
12 frontend/app/layout.tsx · root layout
13 frontend/lib/supabase.ts · Supabase client (auth + realtime)
14 .env.example · all env vars (Supabase URL/key, API keys, Stripe, Resend, R2)
15 Makefile · dev shortcuts

```